SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CSA04 COURSE CODE: Operating Systems

COURSE OUTCOME COVERED

CO1: *Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments.* (BL2, BL3)

QUESTIONS: 1 **Total Marks:100**

Annexure A

Concept Mapping

Code of Conduct:

I Akhilesh Rajesh Uovcue (192124085) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

A.R.
Signature of the student:

95
/100

Annexure A

Concept Mapping

Question 1
Show a multi-level concept map integrating:
Operating System Types → OS Structures → System Calls → OS Functions.
Your map must show cross-links (e.g., how OS structure influences system call implementation). **(BL2)**

Question 2
Construct a comprehensive concept map representing:
Process Management, Memory Management, File System, and I/O System.
Include interdependencies and indicate where system calls interact with each component. **(BL3)**

Question 3
Sketch a comparative concept map for:
Batch OS, Time-Sharing OS, Distributed OS, and Real-Time OS.
Highlight differences, similarities, advantages, and limitations using linking phrases. **(BL2)**

Question 4
Illustrate a dynamic concept map showing the execution flow:
User Program → API → System Call Interface → Kernel → Hardware → Response back to User.
Include feedback loops and control flow paths. **(BL3)**

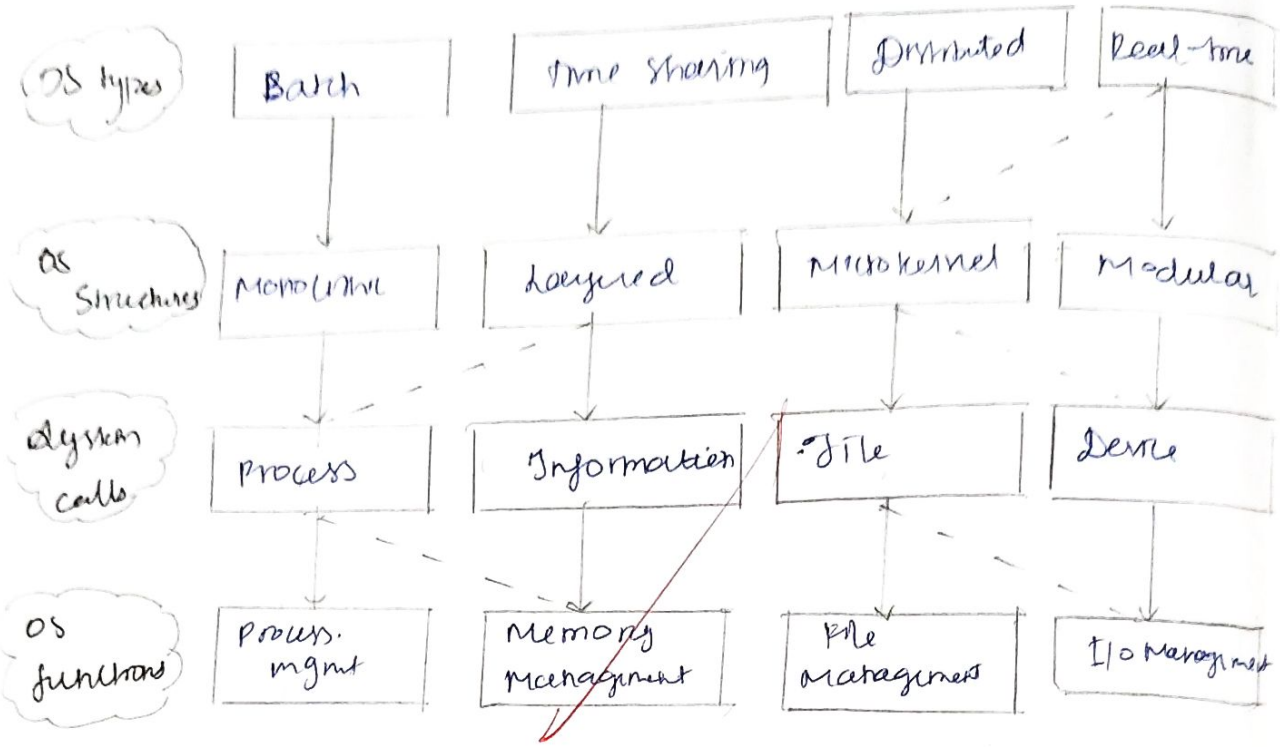
Question 5
Sketch a hierarchical + functional concept map for system calls:
Process, File, Device, and Information system calls.
Also show how they relate to OS services like scheduling, memory allocation, and device handling. **(BL3)**

RUBRICS FOR CALCULATION

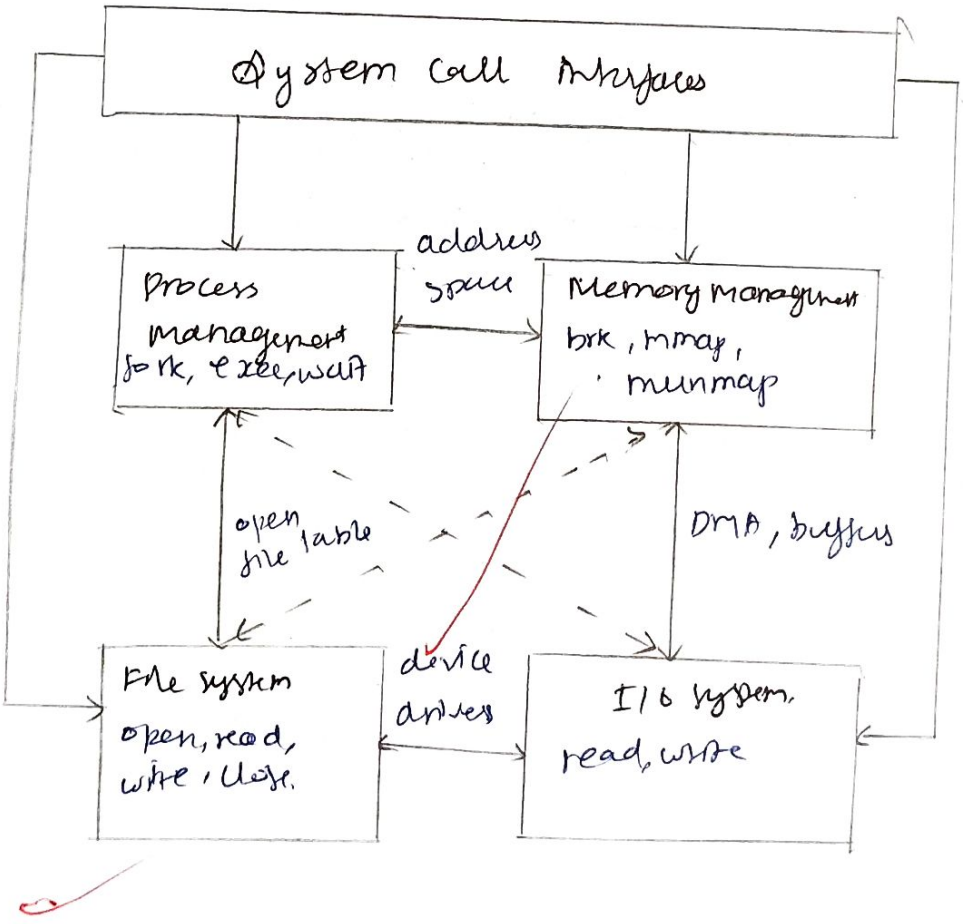
Concept Mapping					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Concept Mapping.

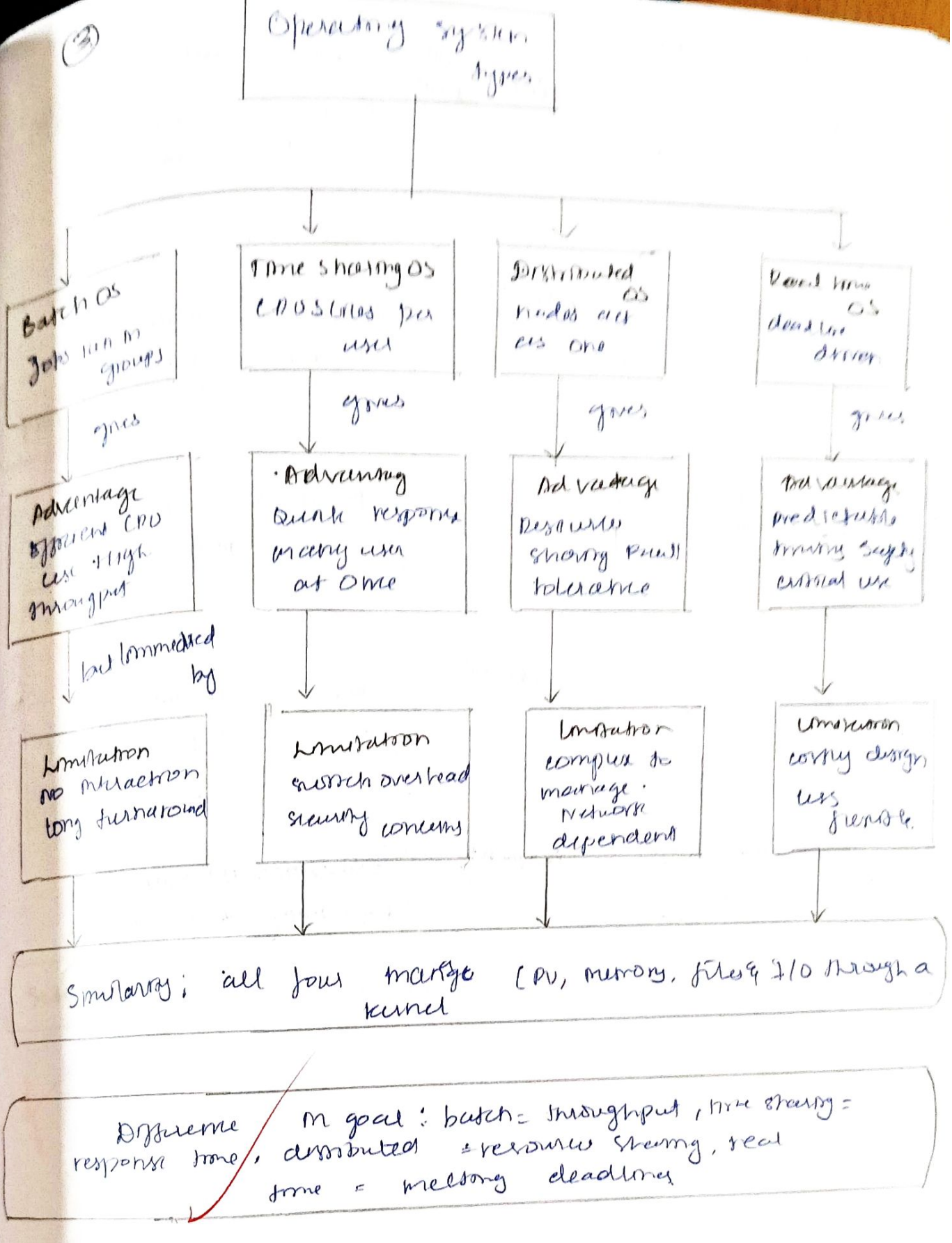
1.)



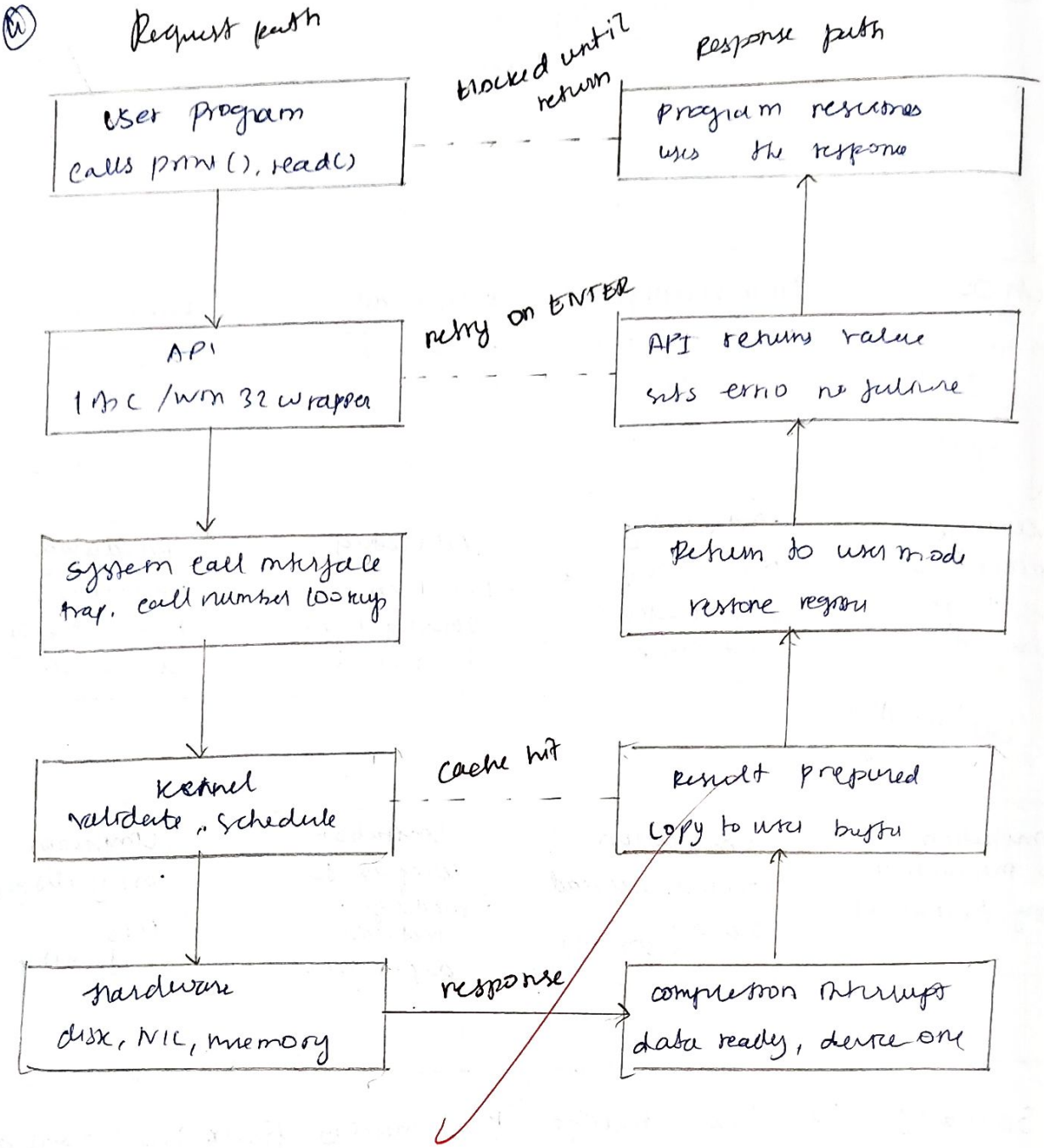
2.

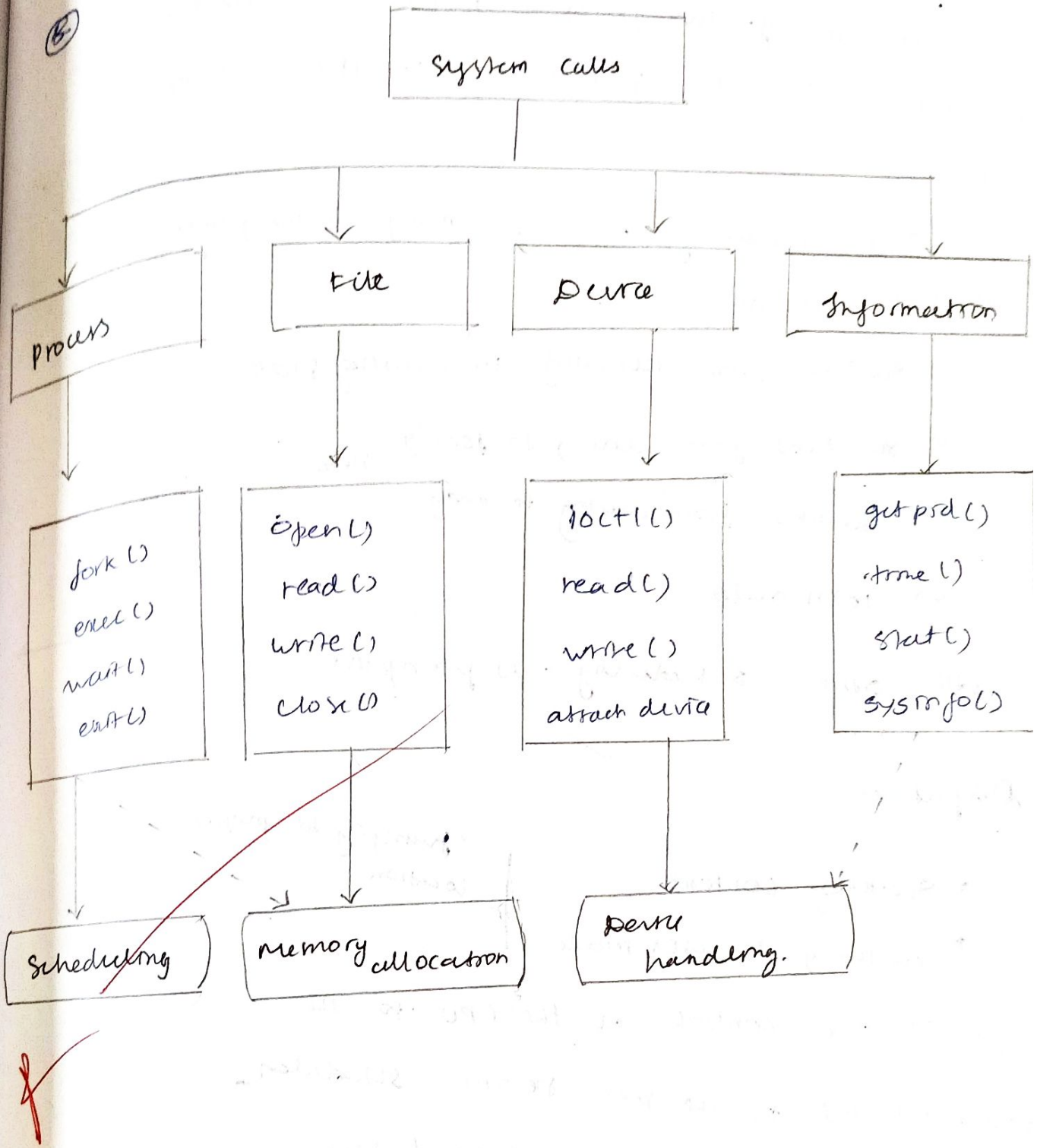


(3)


Operating system types
Batch OS
Jobs run in groups
gives
Advantage
Efficient CPU use + high throughput
but hammered by
Limitation
No interaction
long turnaround
Time sharing OS
CPU shared per user
gives
Advantage
Quick response
many users at once
Limitation
Switch overhead
security concerns
Distributed OS
Nodes act as one
gives
Advantage
Resource sharing
Fault tolerance
Limitation
complex to manage
Network dependent
Real time OS
deadline driven
gives
Advantage
predictable
timing
Safety critical use
Limitation
costly design
less flexible
Similarity: all four manage CPU, memory, files & I/O through a kernel
Difference in goal: batch = throughput, time sharing = response time, distributed = resource sharing, real time = meeting deadlines

(a)


Request path
Response path
User Program
calls printf(), read()
blocked until return
program resumes
uses the response
API
libc / win32 wrapper
retry on ENTER
API returns value
sets errno on failure
system call interface
trap, call number lookup
Return to user mode
restore regs
Kernel
validate, schedule
cache hit
result prepared
copy to user buffer
hardware
disk, NIC, memory
response
completion interrupt
data ready, device done


System calls
process
File
Device
Information
fork()
exec()
wait()
exit()
open()
read()
write()
close()
ioctl()
read()
write()
attach devices
getpid()
alarm()
stat()
sysinfo()
Scheduling
memory allocation
Device handling.